Búsqueda binaria y bisección

Divide y vencerás cuando el espacio está ordenado

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Reconocer la búsqueda binaria y la bisección como algoritmos de divide y vencerás (OA1).
- Plantear la función objetivo de un problema de búsqueda y su formulación recurrente (OA6).
- Implementar por índices la búsqueda binaria y la bisección continua, y justificar su costo $O(\lg n)$ (OA1).
- Transformar un problema de optimización en uno de decisión y resolverlo con búsqueda sobre la respuesta (OA1).

Contenido

1. Repaso: divide y vencerás
2. Buscar en un arreglo ordenado
3. La versión iterativa y sus invariantes
4. Bisección: de arreglos a funciones
5. Búsqueda sobre la respuesta
6. Cómo atacar un problema con bisección
7. Errores comunes
8. Ejercicios
9. Referencias
10. Apéndice: los invariantes, demostrados

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

Dividir, conquistar y combinar

Divide y vencerás (CLRS, Sección 2.3.1)

Dividir el problema en subproblemas del mismo tipo, conquistar cada uno recursivamente hasta un caso base trivial, y combinar las soluciones parciales. El costo queda descrito por una recurrencia

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n),$$

y la partición es **por índices**: cada llamado dice de dónde a dónde trabaja, sin copiar nada.

Lo que dejó el ordenamiento por mezcla

Bajar partiendo, subir mezclando: $T(n) = 2\,T(n/2) + \Theta(n)$, que se resuelve en $\Theta(n \lg n)$. Hoy la recurrencia es más liviana: en cada paso sobrevive *una sola* mitad.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

El juego de adivinar el número

Piense un número entre 1 y 100

Yo solo puedo preguntar "¿es mayor que x?". ¿Cuántas preguntas necesito, en el peor caso, para acertar?

La cuenta

Cada respuesta descarta la mitad de los candidatos:
$100 \to 50 \to 25 \to 13 \to 7 \to 4 \to 2 \to 1$. Siete preguntas bastan, porque $2^7 = 128 \geq 100$. Y para un número entre 1 y un millón alcanzan veinte, porque $2^{20} > 10^6$.

La condición escondida

El juego funciona porque "mayor que x" parte los candidatos en dos bloques ordenados. Esa es la propiedad que hoy vamos a explotar una y otra vez.

El problema

Especificación

Entrada: un arreglo $A[0..N)$ de números ordenado ascendentemente, con $N \geq 1$, y un número v. **Salida**: $\exists\, p \in [0..N).\ A[p] = v$.

Ya sabemos resolverlo... sin usar el orden

La búsqueda lineal revisa las N posiciones una por una: $O(n)$, y su correctitud quedó certificada. Pero ignora por completo que el arreglo viene ordenado.

Pregunta

¿Cómo se juega aquí el juego de adivinar el número?

La función objetivo

Nombrar lo que se busca

Para $0 \leq l < r \leq N$, se define

$$\varphi(l, r): \text{ determina si } v \text{ está en } A[l..r).$$

El intervalo es **medio abierto**: incluye l y excluye r, así que su tamaño es $r - l$.

Reformulación de la salida

La salida pedida es exactamente $\varphi(0, N)$: el problema completo es un caso particular de la función objetivo.

La función objetivo

Con $A = [1, 4, 6, 8, 10, 13, 20, 22]$

$\varphi(0, 8)$ pregunta por todo el arreglo; $\varphi(4, 6)$ pregunta si v está en $[10, 13]$, las posiciones 4 y 5.

El planteamiento recurrente

φ se resuelve partiendo el intervalo

Con $mid = \lfloor (l + r)/2 \rfloor$:

$$\varphi(l, r) = \begin{cases} A[l] = v & \text{si } r - l = 1 \\ \varphi(l, mid) & \text{si } r - l > 1 \text{ y } v < A[mid] \\ \varphi(mid, r) & \text{si } r - l > 1 \text{ y } v \geq A[mid] \end{cases}$$

Los tres momentos

Dividir: calcular *mid*. Conquistar: *un solo* subproblema, la mitad que sobrevive. Combinar: nada, la respuesta del subproblema es la respuesta. Caso base: el intervalo de un elemento.

Por qué descartar la mitad es seguro

Caso $v < A[mid]$: la mitad derecha no sirve

Todo $p \in [mid..r)$ cumple $A[p] \geq A[mid]$ porque el arreglo está ordenado, y $A[mid] > v$: en la mitad derecha no puede haber un testigo. Los testigos posibles, si existen, están todos en $[l..mid)$.

Caso $v \geq A[mid]$: la mitad derecha basta

Aquí v podría estar en las dos mitades, pero no hace falta revisar la izquierda: si hubiera un testigo $p < mid$, el orden da $v = A[p] \leq A[mid] \leq v$, o sea $A[mid] = v$, y el propio mid —que sí queda en $[mid..r)$— es testigo. Buscar solo a la derecha no pierde ninguna respuesta.

Por qué descartar la mitad es seguro

Sin orden no hay descarte

Los dos argumentos usan la monotonía del arreglo. Sobre un arreglo sin ordenar, botar una mitad puede botar la única aparición de v.

La implementación

```
def buscar(lista, v, ini, fin):
    # Determina si v esta en lista[ini..fin), que viene ordenado
    if fin - ini == 1:
        resultado = lista[ini] == v
    else:
        mitad = (ini + fin) // 2
        if v < lista[mitad]:
            resultado = buscar(lista, v, ini, mitad)
        else:
            resultado = buscar(lista, v, mitad, fin)
    return resultado
```

Por índices, como todo el curso

El arreglo completo se consulta con `buscar(lista, v, 0, len(lista))`. Cada llamado recibe de dónde a dónde trabaja y nadie copia nada: dividir cuesta $\Theta(1)$.

Traza

`buscar(lista, 13, 0, 8)` con $A = [1, 4, 6, 8, 10, 13, 20, 22]$

$[ini..fin)$	$mitad$	$A[mitad]$	Comparación	Decisión
[0.,8)	4	10	$13 \geq 10$	mitad derecha
[4.,8)	6	20	$13 < 20$	mitad izquierda
[4.,6)	5	13	$13 \geq 13$	mitad derecha
[5.,6)	—	—	$A[5] = 13$	`True`

De ocho posiciones a una en tres divisiones.

Ahora usted

Corra la traza con $v = 5$. ¿En qué intervalo de un elemento termina y qué responde?

¿Cuánto cuesta?

La recurrencia

Sobrevive una sola mitad y decidir cuál cuesta una comparación:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1), \qquad T(1) = \Theta(1).$$

Expandiendo: cada nivel aporta una constante c, y los niveles se acaban cuando $n/2^k = 1$, o sea $k = \lg n$. El total es $c\,(\lg n + 1)$: $T(n) \in O(\lg n)$.

Y en memoria

La versión recursiva apila un marco por nivel: $O(\lg n)$ de pila. Escrita con un `while` sobre `ini` y `fin` —sin cambiar una línea de la lógica— baja a $O(1)$, porque solo viven tres variables. Para n grande esa diferencia evita desbordar la pila.

¿Cuánto cuesta?

Lineal contra binaria

n	Búsqueda lineal	Búsqueda binaria
10^3	1 000	10
10^6	1 000 000	20
10^9	1 000 000 000	30

Es el juego de adivinar el número, jugado por un algoritmo. CLRS la propone en el Ejercicio 2.3-5 (p. 39), con esta misma cota.

Correctitud

Teorema

Si `lista[ini..fin)` está ordenado y $fin - ini \geq 1$, entonces `buscar(lista, v, ini, fin)` produce el valor de $\exists\, p \in [ini..fin).\ \texttt{lista}[p] = v$.

Demostración: el caso base

Se procede por inducción estructural sobre el tamaño del intervalo, $n = fin - ini$.

Caso base ($n = 1$). El algoritmo devuelve $\texttt{lista}[ini] = v$, y la única posición del intervalo es *ini*: la salida coincide con la especificación. ✓

Correctitud

Demostración: el caso inductivo

Caso inductivo ($n > 1$). Primero, la partición separa y reduce: de $fin - ini \geq 2$ sale $ini < mitad < fin$, así que $[ini..mitad)$ y $[mitad..fin)$ son no vacíos y de tamaño menor que n. Por hipótesis de inducción, el llamado recursivo responde correctamente por su intervalo.

Falta ver que esa respuesta es la del intervalo completo, y ese es justo el argumento del descarte: si $v < A[mitad]$, la mitad derecha no puede tener testigos; si $v \geq A[mitad]$, todo testigo de la izquierda obliga a que $mitad$ mismo sea testigo derecho. En ambos casos, la mitad que sobrevive tiene la misma respuesta que el todo. ✓

Conclusión

Por inducción estructural, la invocación es correcta para todo intervalo; en particular $\varphi(0, N)$, que es la salida pedida. ■

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

El mismo algoritmo, con un ciclo

```
0  def buscar(lista, v):
1      ini = 0
2      fin = len(lista)
3      while fin - ini > 1:
4          mitad = (ini + fin) // 2
5          if v < lista[mitad]:
6              fin = mitad
7          else:
8              ini = mitad
9      return lista[ini] == v
```

La misma lógica, otro certificado

La recursión se demostró por inducción estructural. Esta versión tiene un ciclo, así que le corresponde la otra herramienta: la pareja de invariantes.

Los invariantes

Qué vigila cada uno

Aquí no hay acumulador: lo que cambia es la *ventana* $[ini..fin)$. I_0 acota sus extremos e I_1 dice qué conserva la ventana.

$$I_0 : \quad 0 \leq ini < fin \leq N$$

$$I_1 : \quad \big(\exists\, p \in [0..N).\ A[p] = v\big) \iff \big(\exists\, p \in [ini..fin).\ A[p] = v\big)$$

En palabras: la respuesta del arreglo completo es la misma que la de la ventana. Ese es el trabajo que hace la búsqueda binaria — encoger la ventana sin cambiar la respuesta.

Los invariantes

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación. Las tres partes quedan desarrolladas en el apéndice.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

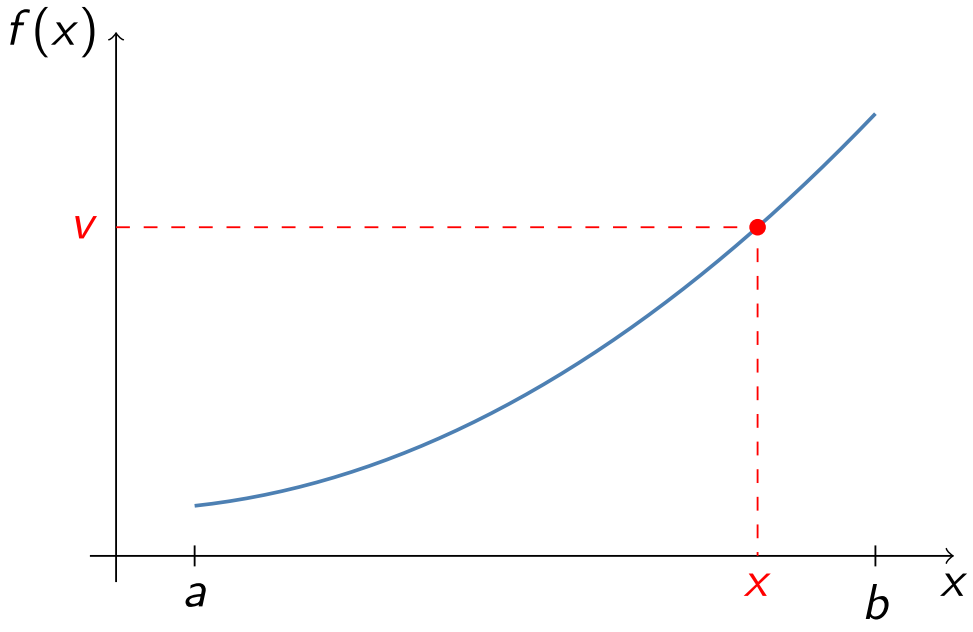
El mismo problema, sin arreglo

Especificación

Entrada: una función f continua y monótona creciente en un intervalo $[a, b]$ y un valor v con $f(a) \leq v \leq f(b)$. **Salida**: $x \in [a, b]$ tal que $f(x) = v$.

La continuidad no es un adorno: es la que garantiza, por el teorema del valor intermedio, que ese x existe. La monotonía es la que permite descartar media.

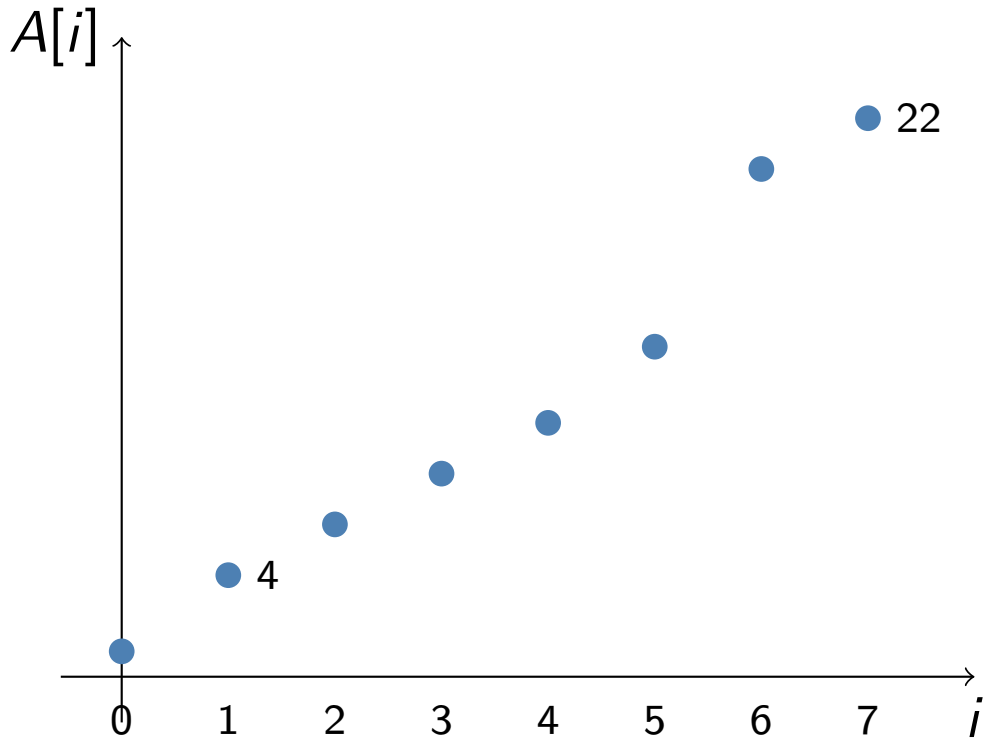
El mismo problema, sin arreglo



La misma jugada

Se evalúa f en la mitad del intervalo. Si $f(mid) < v$, la monotonía garantiza que x está a la derecha; si no, a la izquierda. Una mitad se descarta con certeza, igual que en el arreglo.

El puente entre los dos mundos



El puente entre los dos mundos

Un arreglo ordenado es una función monótona

Graficar $A = [1, 4, 6, 8, 10, 13, 20, 22]$ contra sus posiciones muestra una función $f : \{0, \ldots, 7\} \to \mathbb{Z}$ con $f(i) = A[i]$, creciente. La búsqueda binaria *es* la bisección sobre un dominio discreto, y la bisección es la búsqueda binaria sobre un dominio continuo. Un solo algoritmo, dos presentaciones.

El dominio continuo pide tolerancia

Ya no hay igualdad exacta

Sobre los reales no se puede esperar $f(mid) = v$ al pie de la letra: los `float` redondean y el intervalo nunca queda de "un elemento". La salida pasa a ser una **aproximación**: un x cuyo intervalo de incertidumbre mide menos que una tolerancia ε fijada de antemano.

```
def f(x):
    # La funcion monotona del problema
    return x * x * x + x

def biseccion(v, a, b, eps):
    # Aproxima x en [a, b] con f(x) = v, para f creciente
    while b - a > eps:
        mitad = (a + b) / 2
        if f(mitad) < v:
            a = mitad
        else:
            b = mitad
    return (a + b) / 2
```

Ejemplo trabajado: $x^3 + x = 10$

```
biseccion(10, 0, 3, 1e-6)
```

$f(x) = x^3 + x$ es creciente, $f(0) = 0 \leq 10 \leq f(3) = 30$. Las primeras vueltas:

$[a, b]$	*mitad*	$f(\textit{mitad})$	Decisión
[0, 3]	1,5	4,875	< 10: sube a
[1,5, 3]	2,25	13,64	≥ 10: baja b
[1,5, 2,25]	1,875	8,47	< 10: sube a
[1,875, 2,25]	2,0625	10,83	≥ 10: baja b

El intervalo se cierra alrededor de $x = 2$, donde $2^3 + 2 = 10$.

Ejemplo trabajado: $x^3 + x = 10$

Cuántas vueltas

Cada vuelta parte el intervalo en dos, así que tras k vueltas mide $(b-a)/2^k$. Se necesita $(b-a)/2^k \le \varepsilon$, es decir

$$k = \left\lceil \log_2 \frac{b-a}{\varepsilon} \right\rceil \qquad \text{aquí:} \quad \left\lceil \log_2 \frac{3}{10^{-6}} \right\rceil = 22 \text{ vueltas.}$$

Los invariantes de la bisección

```
0  def biseccion(v, a, b, eps):
1      while b - a > eps:
2          mitad = (a + b) / 2
3          if f(mitad) < v:
4              a = mitad
5          else:
6              b = mitad
7      return (a + b) / 2
```

Qué vigila cada uno

Se escriben a' y b' para los valores originales de a y b, congelados antes de la primera vuelta.

$$I_0: \quad a' \leq a \leq b \leq b' \qquad\qquad I_1: \quad f(a) \leq v \leq f(b)$$

I_0 dice que el intervalo vivo nunca se sale del original ni se da vuelta; I_1, que v sigue *encerrado* entre los dos extremos. Ese encierro es lo que la bisección nunca puede perder.

Los invariantes de la bisección

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación. Las tres partes quedan desarrolladas en el apéndice.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

El problema de la leche

Especificación

Entrada: un arreglo $A[0..N)$ de enteros positivos —los enváses de leche y su contenido— y un número $M \geq 1$ de contenedores disponibles. Los envases se vierten *en orden*, uno tras otro. **Salida**: la capacidad mínima de contenedor con la que basta con M contenedores.

Con $A = [5, 2, 2, 3, 2]$ y $M = 3$

Si los contenedores fueran de capacidad 9: el primero recibe $5 + 2 + 2 = 9$, el segundo $3 + 2 = 5$, y sobra uno. ¿Se puede con contenedores más pequeños? ¿Hasta dónde?

Pregunta

Aquí no hay arreglo ordenado ni función dada. ¿Dónde está lo monótono?

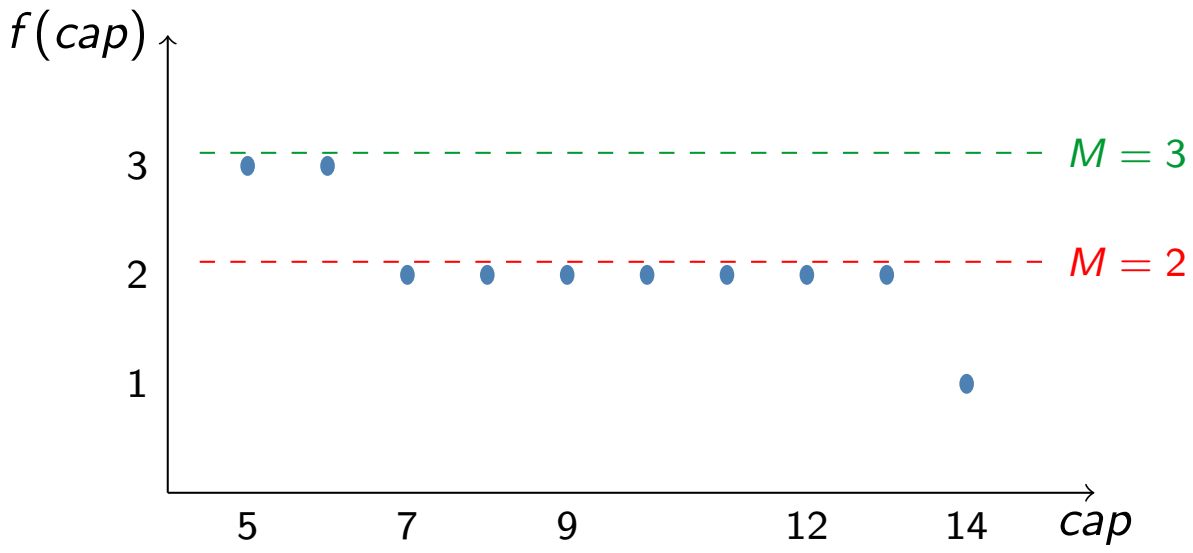
La función objetivo del problema

Nombrarla es la mitad del trabajo

$f(cap)$: contenedores que se necesitan si cada uno tiene capacidad cap.

Calcularla es directo: se vierte envase por envase y, cuando el contenedor actual no da más, se abre otro. Y es **decreciente**: más capacidad nunca exige más contenedores.

La función objetivo del problema



Reformulación de la salida

Se busca el menor *cap* con $f(cap) \leq M$. El rango donde vive la respuesta: de máx(A) —el envase más grande tiene que caber— a $\mathrm{sum}(A)$ —con todo en un contenedor sobra capacidad—. Para el ejemplo, [5.,14].

De optimización a decisión

La transformación

La pregunta difícil "¿cuál es la capacidad óptima?" se cambia por la pregunta fácil "¿alcanza con capacidad *cap*?". La segunda se responde con un recorrido lineal, y como f es monótona, las respuestas forman dos bloques: primero los *cap* que no alcanzan, luego los que sí. El borde entre bloques es el óptimo, y un borde entre dos bloques ordenados se encuentra con búsqueda binaria.

El patrón, en general

Siempre que un problema de optimización admita una pregunta de decisión monótona en el parámetro que se optimiza, la respuesta se puede buscar binariamente sobre ese parámetro. En los jueces en línea el patrón aparece por todas partes.

La implementación

```
def contenedores(envases, cap):
    # Contenedores de capacidad cap que exigen los envases, en orden
    cuenta = 1
    acumulado = 0
    i = 0
    while i < len(envases):
        if acumulado + envases[i] <= cap:
            acumulado = acumulado + envases[i]
        else:
            cuenta = cuenta + 1
            acumulado = envases[i]
        i = i + 1
    return cuenta
```

La implementación

```
def capacidad_minima(envases, m):
    # Menor capacidad de contenedor con la que bastan m contenedores
    a = max(envases)
    b = sum(envases)
    while a < b:
        mitad = (a + b) // 2
        if contenedores(envases, mitad) <= m:
            b = mitad
        else:
            a = mitad + 1
    return a
```

La implementación, leída con calma

El chequeo

`contenedores` es la función objetivo hecha código: un recorrido $\Theta(n)$ que abre contenedor nuevo cuando el actual no da más.

La búsqueda

`capacidad_minima` busca el borde: si con `mitad` alcanza, la respuesta es `mitad` o algo menor (baja b); si no alcanza, es estrictamente mayor (sube a hasta $\mathit{mitad} + 1$). Al cerrarse el intervalo, a es el menor valor que alcanza.

El costo

Cada chequeo cuesta $\Theta(n)$ y hay $O(\lg(\mathrm{sum}(A)))$ chequeos: $O(n \lg(\mathrm{sum}(A)))$ en total. La búsqueda binaria corre sobre el rango de *respuestas*, no sobre el arreglo.

Traza

`capacidad_minima([5, 2, 2, 3, 2], 3)`

$[a..b]$	*mitad*	$f(\mathit{mitad})$	¿≤ 3?	Decisión
[5.,14]	9	2	sí	$b = 9$
[5.,9]	7	2	sí	$b = 7$
[5.,7]	6	3	sí	$b = 6$
[5.,6]	5	3	sí	$b = 5$

El intervalo se cierra en $a = b = 5$: con capacidad 5 alcanza, y menos no se puede porque el envase de 5 tiene que caber.

Ahora usted

Repita la traza con $M = 2$. La primera diferencia aparece en *mitad* $= 6$: ¿qué decide el algoritmo ahí y en qué respuesta termina?

Los invariantes de la búsqueda sobre la respuesta

```
10  def capacidad_minima(envases, m):
11      a = max(envases)
12      b = sum(envases)
13      while a < b:
14          mitad = (a + b) // 2
15          if contenedores(envases, mitad) <= m:
16              b = mitad
17          else:
18              a = mitad + 1
19      return a
```

Quién es el óptimo

Sea $R = \{\, cap : f(cap) \leq M \,\}$ el conjunto de capacidades viables y $cap^* = \text{mín}\, R$ la respuesta pedida. Como f es decreciente, R es *cerrado hacia arriba*: si una capacidad alcanza, cualquiera mayor también. De ahí que su complemento sea cerrado hacia abajo, hecho que se usa en la estabilidad.

Los invariantes de la búsqueda sobre la respuesta

Qué vigila cada uno

Lo que cambia es la ventana de candidatos $[a..b]$, y lo que hay que conservar es que el óptimo siga adentro.

$$I_0: \quad \text{máx}(A) \leq a \leq b \leq \text{sum}(A) \qquad\qquad I_1: \quad a \leq cap^* \leq b$$

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación. Las tres partes quedan desarrolladas en el apéndice.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

La metodología, en tres pasos

1. Identificar el parámetro

¿Qué número es el que hay que hallar? En la leche, la capacidad; en el arreglo ordenado, la posición. Ese número, y no otro, es sobre el que se va a buscar.

2. Definir la función y comprobar que es monótona

Se escribe f del parámetro y se dice *qué significa*. Casi siempre responde una pregunta de viabilidad: dado un valor, ¿alcanza? Y se argumenta por qué crece o decrece: si f no es monótona, la bisección no aplica y hay que buscar otra formulación.

La metodología, en tres pasos

3. Acotar el rango y buscar

Se justifican las dos cotas: un valor que seguro no alcanza y uno que seguro sí. Entre ellos, la bisección encuentra el borde en $O(\lg(\text{ancho del rango}))$ evaluaciones de f.

La señal que hay que aprender a ver

Cuando el enunciado pide *el mínimo* o *el máximo* de algo y verificar un valor concreto es fácil, casi siempre hay una bisección escondida.

Detalles de implementación que cuestan puntos

Entero o continuo, dos plantillas distintas

Con enteros, el ciclo es `while a < b` y la rama que descarta debe escribir `a = mitad + 1`; con `a = mitad` el intervalo puede quedarse quieto y el programa se cuelga. Con reales, el ciclo es `while b - a > eps` y las dos ramas asignan `mitad` sin sumar nada.

Cuánto vale ε

Lo dice el enunciado: si pide cuatro decimales, $\varepsilon = 10^{-6}$ sobra y cuesta apenas unas vueltas más. Un ε más fino que la precisión del `double` deja el ciclo girando sin avanzar.

Detalles de implementación que cuestan puntos

El costo total

Es el costo de evaluar f multiplicado por el número de vueltas. Si f cuesta $\Theta(n)$ y el rango tiene ancho R, el total es $O(n \lg R)$: por eso conviene que la verificación sea barata.

Antes de enviar

Corra el ejemplo del enunciado a mano y pruebe los extremos: el rango de tamaño 1, el caso donde la respuesta es la cota inferior y aquel donde es la superior.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

Errores comunes

Dónde se caen estos algoritmos

- **Un intervalo que no progresa**: con división entera, actualizar con $a = \mathit{mitad}$ en vez de $a = \mathit{mitad} + 1$ puede dejar el intervalo idéntico y el ciclo no termina. La terminación exige que *cada* rama reduzca.
- **Descartar sin monotonía**: sobre un arreglo sin ordenar o una función que sube y baja, el descarte puede botar la única solución. La monotonía no se asume: se argumenta, como con $f(\mathit{cap})$.
- **Un rango inicial que no contiene la respuesta**: si la respuesta puede ser $\max(A)$ y el rango arranca en $\max(A) + 1$, ninguna búsqueda la encuentra. Las cotas del rango se justifican, no se adivinan.

Errores comunes

Y dos más, del lado continuo

- **Suponer que la respuesta existe**: si v queda fuera de $[f(a), f(b)]$, el algoritmo no avisa: converge al extremo del intervalo cuyo valor de f es el más cercano a v. Al terminar hay que comprobar que el candidato de verdad cumple lo pedido.
- **Tolerancia mal escogida**: un ε más fino que la precisión del `float` deja el ciclo dando vueltas sin que $b - a$ baje. La tolerancia se fija según lo que pide el problema.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

Para practicar el método

Sobre lo visto hoy

1. Escriba la recurrencia de la búsqueda binaria y justifique la cota $\Theta(\lg n)$ del peor caso (CLRS, Ejercicio 2.3-5, p. 39).
2. Modifique `buscar` para que devuelva la *posición* de v, o -1 si no está. ¿Qué cambia en la función objetivo y en el caso base?
3. Reescriba `buscar` con un `while` en vez de recursión, sin cambiar la lógica. Compare el consumo de pila de las dos versiones.
4. Para el ciclo de `biseccion`: proponga el invariante que garantiza que la respuesta nunca se sale de $[a, b]$ y úselo para argumentar la correctitud.
5. Halle la raíz de $f(x) = x^3 - 2x - 5$ en $[2, 3]$ con $\varepsilon = 10^{-3}$. Antes de programar, prediga el número de vueltas con la fórmula del logaritmo; deberían ser 10. La raíz anda cerca de 2,0946.

Problemas de juez en línea

UVa 11909 — Soya Milk

Enunciado: `https://vjudge.net/problem/UVA-11909`
Enviar: `https://vjudge.net/problem/UVA-11909`

Una caja de leche de dimensiones $l \times w \times h$ se inclina un ángulo θ; hay que hallar el volumen que queda adentro. **Pista**: la geometría da dos casos según el líquido toque o no el borde superior, y en cada uno el volumen sale de una longitud desconocida. Esa longitud es el parámetro: la función que la relaciona con el dato conocido es creciente, así que se halla con bisección continua.

Problemas de juez en línea

UVa 11646 — Athletics Track

Enunciado: `https://vjudge.net/problem/UVA-11646`
Enviar: `https://vjudge.net/problem/UVA-11646`

Una pista de dos rectas y dos arcos, con proporción largo : ancho dada, debe medir 400 metros. **Pista**: el parámetro es el largo l; el ancho sale de la proporción y el perímetro $f(l)$ crece con l. Se busca l con $f(l) = 400$: bisección continua sobre un rango generoso.

Problemas de juez en línea

UVa 714 — Copying Books

Enunciado: `https://vjudge.net/problem/UVA-714`
Enviar: `https://vjudge.net/problem/UVA-714`

K escribas copian libros consecutivos; hay que minimizar las páginas del escriba más cargado. **Pista**: es la leche con otro disfraz. $f(t) =$ escribas necesarios si cada uno copia a lo sumo t páginas, decreciente, y el rango va de máx(A) a $\mathrm{sum}(A)$. Cuidado con la salida: el juez pide además la partición, y entre varias válidas exige la que carga más a los últimos escribas.

Para practicar en el navegador

Los mismos pasos, con respuesta inmediata

`buscar` recorre la ventana división por división y lleva los dos invariantes hasta la demostración completa; `bisección` muestra el intervalo cerrándose sobre tres funciones distintas, con la cuenta de vueltas y la cota del error.

Para practicar en el navegador

Y un problema de juez, paso a paso

Un recorrido guiado sobre *UVa 10341 — Solve It*, que no aparece en estas diapositivas: entender la entrada, nombrar la función objetivo, **demostrar** la monotonía término a término, mirar el rango, decidir si la solución existe, fijar la tolerancia y solo entonces escribir código. La función de ese problema *decrece*, así que copiar la plantilla de hoy sin invertir la comparación da un programa que corre y responde mal.

Enunciado: `https://vjudge.net/problem/UVA-10341`
Enviar: `https://vjudge.net/problem/UVA-10341`

Dónde

`https://cardel.github.io/notasUniversidad/2026-II/AyG%20PUJ/C3/Ejercicios/`

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

Referencias

- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, MIT Press, 2009. Sección 2.3 y Ejercicio 2.3-5, p. 39.
- S. Halim, F. Halim y S. Effendy. *Competitive Programming 4*, 2018. Capítulo 3: búsqueda binaria sobre la respuesta.
- C. Rocha. *Diseño y Análisis de Algoritmos*.

Contenido

1 Repaso: divide y vencerás

2 Buscar en un arreglo ordenado

3 La versión iterativa y sus invariantes

4 Bisección: de arreglos a funciones

5 Búsqueda sobre la respuesta

6 Cómo atacar un problema con bisección

7 Errores comunes

8 Ejercicios

9 Referencias

10 Apéndice: los invariantes, demostrados

Búsqueda binaria: inicialización

Inicialización

Por las líneas 1–2, $ini = 0$ y $fin = N$.

Para I_0: se pide $0 \leq 0 < N \leq N$. La desigualdad estricta $0 < N$ vale por la precondición $N \geq 1$; las otras dos son igualdades. ✓

Para I_1: sustituyendo $ini = 0$ y $fin = N$, el lado derecho queda $\exists\, p \in [0..N).\ A[p] = v$, que es literalmente el lado izquierdo. Una equivalencia entre una fórmula y ella misma es verdadera. ✓

Búsqueda binaria: la partición separa

Lo que se asume

Se considera una iteración arbitraria con $ini = a$ y $fin = b$, diferente a la última; como el cuerpo se ejecuta, $b - a > 1$, o sea $b \geq a + 2$. Se asumen $0 \leq a < b \leq N$ y la equivalencia de I_1 para la ventana $[a..b)$.

Primero: $a < mitad < b$

La línea 4 calcula $mitad = \lfloor (a + b)/2 \rfloor$, y hay que verificar que cae *estrictamente* adentro, porque de eso dependen las cotas y el progreso.

- De $b \geq a + 2$ sale $a + b \geq 2a + 2$, luego $mitad \geq \lfloor (2a + 2)/2 \rfloor = a + 1 > a$.
- De $a \leq b - 2$ sale $a + b \leq 2b - 2$, luego $mitad \leq \lfloor (2b - 2)/2 \rfloor = b - 1 < b$.

Con eso, *mitad* es una posición válida del arreglo y las dos ventanas candidatas son no vacías.

Búsqueda binaria: estabilidad, los dos casos

Caso $v < A[mitad]$ — la línea 6 deja $fin = mitad$

I_0 evaluado en los nuevos valores exige $0 \leq a < mitad \leq N$: la primera vale por lo asumido, $a < mitad$ se acaba de probar y $mitad < b \leq N$. ✓

I_1 evaluado en los nuevos valores exige que la respuesta total equivalga a la de $[a..mitad)$. Por lo asumido equivale a la de $[a..b)$, así que basta ver que $[mitad..b)$ no aporta testigos: para todo p en ese rango, $A[p] \geq A[mitad]$ porque el arreglo está ordenado, y $A[mitad] > v$, luego $A[p] \neq v$. ✓

Búsqueda binaria: estabilidad, los dos casos

Caso $v \geq A[mitad]$ — la línea 8 deja $ini = mitad$

I_0 exige $0 \leq mitad < b \leq N$: $mitad \geq a + 1 \geq 1 > 0$ y $mitad < b$ ya están probados. ✓

I_1 exige que la respuesta de $[a..b)$ equivalga a la de $[mitad..b)$. De derecha a izquierda es inmediato, porque $[mitad..b) \subseteq [a..b)$. De izquierda a derecha: si hubiera un testigo $p \in [a..mitad)$, de $p < mitad$ y el orden sale $v = A[p] \leq A[mitad]$, que junto con $v \geq A[mitad]$ da $A[mitad] = v$; entonces $mitad$ —que sí pertenece a $[mitad..b)$— también es testigo. ✓

Conclusión del paso

En los dos casos valen I_0 e I_1 con los valores nuevos. Por lo tanto, los invariantes son estables.

Búsqueda binaria: terminación

El ciclo termina

El ancho $fin - ini$ es un entero positivo por I_0. En el primer caso pasa a valer $mitad - a$ y en el segundo $b - mitad$; como $a < mitad < b$, los dos son al menos 1 y estrictamente menores que $b - a$. Un entero positivo que decrece estrictamente no puede hacerlo para siempre.

Con qué valores termina

Al salir, la condición $fin - ini > 1$ es falsa, o sea $fin - ini \leq 1$; e I_0 da $ini < fin$, o sea $fin - ini \geq 1$. El único valor que cumple las dos cosas es $fin - ini = 1$, es decir $fin = ini + 1$: la ventana quedó en una sola posición.

Búsqueda binaria: terminación

Qué entregan los invariantes

Sustituyendo en I_1, el lado derecho es $\exists\, p \in [ini..ini{+}1).\ A[p] = v$, un rango de una posición, que vale exactamente $A[ini] = v$. Entonces la respuesta del arreglo completo es $A[ini] = v$. Por lo tanto, los invariantes son correctos. ■

Búsqueda binaria: el teorema del algoritmo

Teorema 2

La invocación `buscar(lista, v)` sobre un arreglo ordenado de tamaño $N \geq 1$ produce el valor de $\exists\, p \in [0..N).\ \texttt{lista}[p] = v$.

Demostración: es trivial a partir de la correctitud de I_0 e I_1 (Teorema 1): la terminación deja $\mathit{fin} = \mathit{ini} + 1$, y sustituir ese valor en I_1 dice que la respuesta pedida es $A[\mathit{ini}] = v$, que es justo lo que retorna la línea 9. ■

De paso, la memoria

Esta versión no apila llamadas: sobreviven `ini`, `fin` y `mitad`. El costo en espacio baja de $O(\lg n)$ a $O(1)$ sin tocar la lógica.

Bisección: inicialización y estabilidad

Inicialización

Antes de la primera vuelta $a = a'$ y $b = b'$. Para I_0: $a' \leq a' \leq b' \leq b'$, donde la de la mitad vale porque $[a', b']$ es un intervalo. ✓ Para I_1: sustituyendo queda $f(a') \leq v \leq f(b')$, que es exactamente la precondición. ✓

Bisección: inicialización y estabilidad

Estabilidad

Iteración arbitraria, diferente a la última: como el cuerpo se ejecuta, $b - a > \varepsilon > 0$, luego $a < b$ y por lo tanto $a < \mathit{mitad} < b$, con $\mathit{mitad} = (a + b)/2$. Se asumen I_0 e I_1. La línea 3 abre dos casos:

- $f(\mathit{mitad}) < v$: la línea 4 deja $a = \mathit{mitad}$. I_0 evaluado ahí exige $a' \leq \mathit{mitad} \leq b \leq b'$: la primera sale de $a' \leq a < \mathit{mitad}$ y las otras de $\mathit{mitad} < b \leq b'$. ✓ I_1 exige $f(\mathit{mitad}) \leq v \leq f(b)$: la izquierda es el caso mismo y la derecha es la que se asumió, porque b no cambió. ✓
- $f(\mathit{mitad}) \geq v$: la línea 6 deja $b = \mathit{mitad}$. I_0 exige $a' \leq a \leq \mathit{mitad} \leq b'$: sale de $a < \mathit{mitad} < b \leq b'$. ✓ I_1 exige $f(a) \leq v \leq f(\mathit{mitad})$: la izquierda es la que se asumió, porque a no cambió, y la derecha es el caso mismo. ✓

Por lo tanto, los invariantes son estables.

Bisección: terminación y teorema

Terminación

Cada vuelta deja el ancho en exactamente la mitad: de $b - a$ pasa a $\mathit{mitad} - a = (b-a)/2$ o a $b - \mathit{mitad} = (b-a)/2$. Tras k vueltas vale $(b' - a')/2^k$, que baja de cualquier $\varepsilon > 0$; el ciclo termina, y lo hace en $\lceil \log_2((b' - a')/\varepsilon) \rceil$ vueltas.

Al salir, la condición $b - a > \varepsilon$ es falsa, o sea $b - a \leq \varepsilon$; e I_0 da $a \leq b$, o sea $b - a \geq 0$. Intersectando: $0 \leq b - a \leq \varepsilon$.

Qué entregan los invariantes

I_1 dice $f(a) \leq v \leq f(b)$, así que por el teorema del valor intermedio existe una solución $x \in [a, b]$ de $f(x) = v$. El valor retornado, $\hat{x} = (a+b)/2$, también está en $[a, b]$, y dos puntos de un intervalo de ancho a lo sumo ε distan a lo sumo eso; midiendo desde el centro, $|\hat{x} - x| \leq \varepsilon/2$. Por lo tanto, los invariantes son correctos. ■

Bisección: terminación y teorema

Teorema 2

La invocación `biseccion(v, a, b, eps)` sobre una f continua y creciente con $f(a) \leq v \leq f(b)$ produce un $\hat{x}$ que dista a lo sumo $\varepsilon/2$ de una solución de $f(x) = v$.

Demostración: es trivial a partir de la correctitud de I_0 e I_1 (Teorema 1): la terminación deja un intervalo de ancho a lo sumo ε que sigue encerrando a v, y de ahí sale la cota del error del valor que retorna la línea 7. ■

La diferencia con el caso discreto

En el arreglo, la terminación entrega la respuesta *exacta*. Aquí entrega una respuesta con error acotado, y esa cota es parte del enunciado del teorema: sobre los reales no hay otra cosa que prometer.

Búsqueda sobre la respuesta: inicialización

Inicialización

Las líneas 1–2 dejan $a = \text{máx}(A)$ y $b = \text{sum}(A)$.

Para I_0: se pide $\text{máx}(A) \leq \text{máx}(A) \leq \text{sum}(A) \leq \text{sum}(A)$; la del medio vale porque los envases son enteros positivos, así que la suma de todos no baja del mayor. ✓

Para I_1: hay que ver que cap^* arranca dentro del rango, y son dos cosas distintas. Que $cap^* \leq \text{sum}(A)$: con esa capacidad todo cabe en un contenedor, $f(\text{sum}(A)) = 1 \leq M$, luego $\text{sum}(A) \in R$ y el mínimo de R no lo supera. Que $cap^* \geq \text{máx}(A)$: una capacidad menor que el envase más grande no puede recibirlo en ningún contenedor, así que ninguna está en R. ✓

Búsqueda sobre la respuesta: estabilidad

Lo que se asume

Iteración arbitraria, diferente a la última: como el cuerpo se ejecuta, $a < b$. Se asumen I_0 e I_1.

Primero: $a \leq \mathit{mitad} < b$

Con $\mathit{mitad} = \lfloor (a+b)/2 \rfloor$ y $b \geq a+1$: de $a+b \geq 2a+1$ sale $\mathit{mitad} \geq \lfloor (2a+1)/2 \rfloor = a$; de $a+b \leq 2b-1$ sale $\mathit{mitad} \leq \lfloor (2b-1)/2 \rfloor = b-1 < b$.

Búsqueda sobre la respuesta: estabilidad

Los dos casos

- $f(\mathit{mitad}) \leq M$: la línea 6 deja $b = \mathit{mitad}$. Aquí $\mathit{mitad} \in R$, luego $\mathit{cap}^* \leq \mathit{mitad}$ por ser el mínimo. I_0 exige $\text{máx}(A) \leq a \leq \mathit{mitad} \leq \text{sum}(A)$, que sale de $a \leq \mathit{mitad} < b \leq \text{sum}(A)$. ✓ I_1 exige $a \leq \mathit{cap}^* \leq \mathit{mitad}$: la izquierda es la asumida y la derecha se acaba de probar. ✓
- $f(\mathit{mitad}) > M$: la línea 8 deja $a = \mathit{mitad} + 1$. Aquí $\mathit{mitad} \notin R$, y como el complemento de R es cerrado hacia abajo, ninguna capacidad $\leq \mathit{mitad}$ está en R; en particular $\mathit{cap}^* > \mathit{mitad}$, o sea $\mathit{cap}^* \geq \mathit{mitad} + 1$. I_0 exige $\text{máx}(A) \leq \mathit{mitad} + 1 \leq b \leq \text{sum}(A)$: la izquierda sale de $\mathit{mitad} + 1 > a \geq \text{máx}(A)$ y la del medio de $\mathit{mitad} < b$. ✓ I_1 exige $\mathit{mitad} + 1 \leq \mathit{cap}^* \leq b$: la izquierda se acaba de probar y la derecha es la asumida. ✓

Por lo tanto, los invariantes son estables.

Búsqueda sobre la respuesta: terminación y teorema

Terminación

El ancho $b - a$ es un entero no negativo por I_0. En el primer caso pasa a $mitad - a$, que es $\leq b - 1 - a < b - a$; en el segundo, a $b - mitad - 1$, que es $\leq b - a - 1 < b - a$. Un entero no negativo que decrece estrictamente no puede hacerlo para siempre.

Al salir, la condición $a < b$ es falsa, o sea $a \geq b$; e I_0 da $a \leq b$. El único valor que cumple las dos cosas es $a = b$.

Qué entregan los invariantes

Sustituyendo $a = b$ en I_1 queda $a \leq cap^* \leq a$, es decir $cap^* = a$: la ventana se cerró exactamente sobre el óptimo. Por lo tanto, los invariantes son correctos. ■

Búsqueda sobre la respuesta: terminación y teorema

Teorema 2

La invocación `capacidad_minima(envases, m)` sobre un arreglo de enteros positivos y $M \geq 1$ produce la menor capacidad con la que bastan M contenedores.

Demostración: es trivial a partir de la correctitud de I_0 e I_1 (Teorema 1): la terminación deja $a = b$ y sustituir en I_1 da $a = cap^*$, que es lo que retorna la línea 9. ■

Los tres invariantes dicen lo mismo

Los tres ciclos de hoy tienen el mismo I_1 con distinto disfraz: *lo que se busca sigue dentro de la ventana*. En el arreglo era la respuesta al existencial, en la bisección el valor v encerrado entre $f(a)$ y $f(b)$, y aquí el óptimo cap^*. Al resolver un problema nuevo, ese es el invariante que hay que escribir.